

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



Laboratorio 01

Curso:

Computación Gráfica

DOCENTE:

René Nieto

PRESENTADO POR:

Puma Larico, Cristhian Willians

AREQUIPA – PERÚ

2025

ÍNDICE

1. Introducción:	2
2. Desarrollo de la Aplicación:	2
2.1. Escenario y Modelado	2
2.2. Implementación de la Física y Movimiento	3
3. Código Fuente y Funcionamiento:	3
3.1. MovimientoCañon.cs	3
3.2. Proyectoil.cs	5
3.3. LogicaJuego.cs	6
4. Gestión de Colisiones y UI:	6
4.1. Detección	7
4.2. Interfaz de Usuario (UI)	7
5. Resultados y Conclusiones:	7
6. Referencias:	8

Link del video:

 puy-gsrk-tgg (2026-05-03 17:31 GMT-5)

Link de github:

<https://github.com/c-puma/CGVCM-Lab01/tree/master>

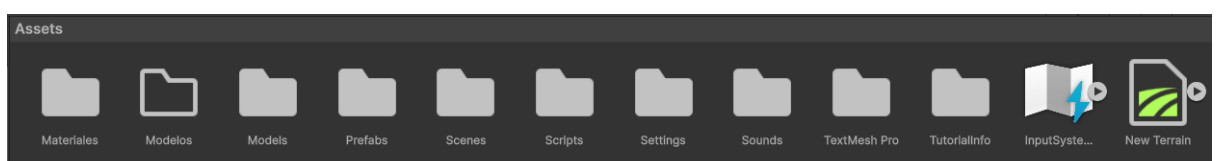
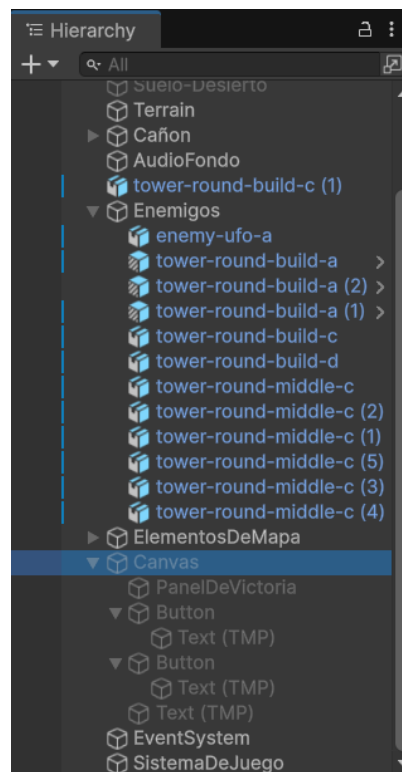
1. Introducción:

Este laboratorio tiene como fin desarrollar una aplicación multimedia que integre un motor de física para la simulación en tiempo real de lanzamiento de proyectiles

2. Desarrollo de la Aplicación:

2.1. Escenario y Modelado

Se diseñó un entorno desértico con estética Low Poly para optimizar el performance y mantener altos FPS. Se integraron modelos de ruinas y ambientación desértica, organizados en una jerarquía clara (Entorno, Jugador, Sistema).



2.2. Implementación de la Física y Movimiento

El personaje (cañón) permite rotación horizontal controlada por el usuario mediante las teclas A/D (Input horizontal). El disparo utiliza el método `AddForce` con el modo `ForceMode.Impulse`, simulando una descarga de energía instantánea sobre la bala.

3. Código Fuente y Funcionamiento

Se desarrollaron tres scripts principales:

- 3.1. `MovimientoCañon.cs`: Gestiona la rotación, el instanciado de proyectiles y la reproducción de sonidos al disparar los proyectiles para simular un entorno más inmersivo.

C/C++

```
using UnityEngine;

public class MovimientoCañon : MonoBehaviour
{
    public GameObject prefabBala;
    public Transform puntoDisparo;
    public float fuerzaDisparo = 500f;
    public float velocidadRotacion = 100f;

    public AudioClip sonidoDisparo;
    private AudioSource audioSource;

    void Start()
    {
        // Obtenemos el componente AudioSource al iniciar
        audioSource = GetComponent<AudioSource>();
    }

    void Update()
    {
        float rotarH = Input.GetAxis("Horizontal") * velocidadRotacion *
Time.deltaTime;
        transform.Rotate(0, rotarH, 0);

        if (Input.GetKeyDown(KeyCode.Space))
        {
            Disparar();
        }
    }
}
```

```

void Disparar()
{
    // Instancia el proyectil
    GameObject bala = Instantiate(prefabBala, puntoDisparo.position,
    puntoDisparo.rotation);

    // Obtiene el Rigidbody para aplicar física
    Rigidbody rb = bala.GetComponent<Rigidbody>();

    if (rb != null)
    {
        // Aplica una fuerza hacia adelante
        rb.AddForce(puntoDisparo.forward * fuerzaDisparo,
        ForceMode.Impulse);
        // REPRODUCIR SONIDO
        if (audioSource != null && sonidoDisparo != null)
        {
            audioSource.PlayOneShot(sonidoDisparo);
        }
    }
}

```

- 3.2. Proyectil.cs: Controla el ciclo de vida del proyectil para optimizar recursos y detecta colisiones con objetos etiquetados como "Target".

C/C++

```

using UnityEngine;

public class Proyectil : MonoBehaviour
{
    public float tiempoVida = 5f; // Para no saturar el motor físico

    void Start()
    {
        // Destruye la bala después de unos segundos
        Destroy(gameObject, tiempoVida);
    }

    private void OnCollisionEnter(Collision collision)
    {
        // Detecta si golpeó a un enemigo o un aro (usando Tags)
        if (collision.gameObject.CompareTag("Target"))

```

```

    {
        Debug.Log(";Impacto certero!");
        LogicaJuego logica = FindFirstObjectByType<LogicaJuego>();
        if (logica != null) logica.GanarPuntos(10);

        Destroy(gameObject); // Destruye la bala al impactar
    }

    // Si toca el suelo, simplemente puede revotar para simular la
    fisica real
}

}

```

- 3.3. LogicaJuego.cs: Gestiona el sistema de puntuación y activa la pantalla de victoria al alcanzar los 100 puntos.

C/C++

```

using UnityEngine;
using UnityEngine.SceneManagement; // Para reiniciar el nivel

public class LogicaJuego : MonoBehaviour
{
    public int puntosParaGanar = 100;
    public int puntosActuales = 0;
    public GameObject panelVictoria;
    // Método para sumar puntos
    public void GanarPuntos(int cantidad)
    {
        puntosActuales += cantidad;

        if (puntosActuales >= puntosParaGanar)
        {
            MostrarVictoria();
        }
    }

    void MostrarVictoria()
    {
        panelVictoria.SetActive(true); // Muestra el panel
        Time.timeScale = 0f;
        Cursor.lockState = CursorLockMode.None; // Libera el mouse para
        el botón
        Cursor.visible = true;
    }
}

```

```

    }

    // Función para el botón de Reiniciar
    public void ReiniciarJuego()
    {
        Time.timeScale = 1f; // Reanuda el tiempo
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }
}

```

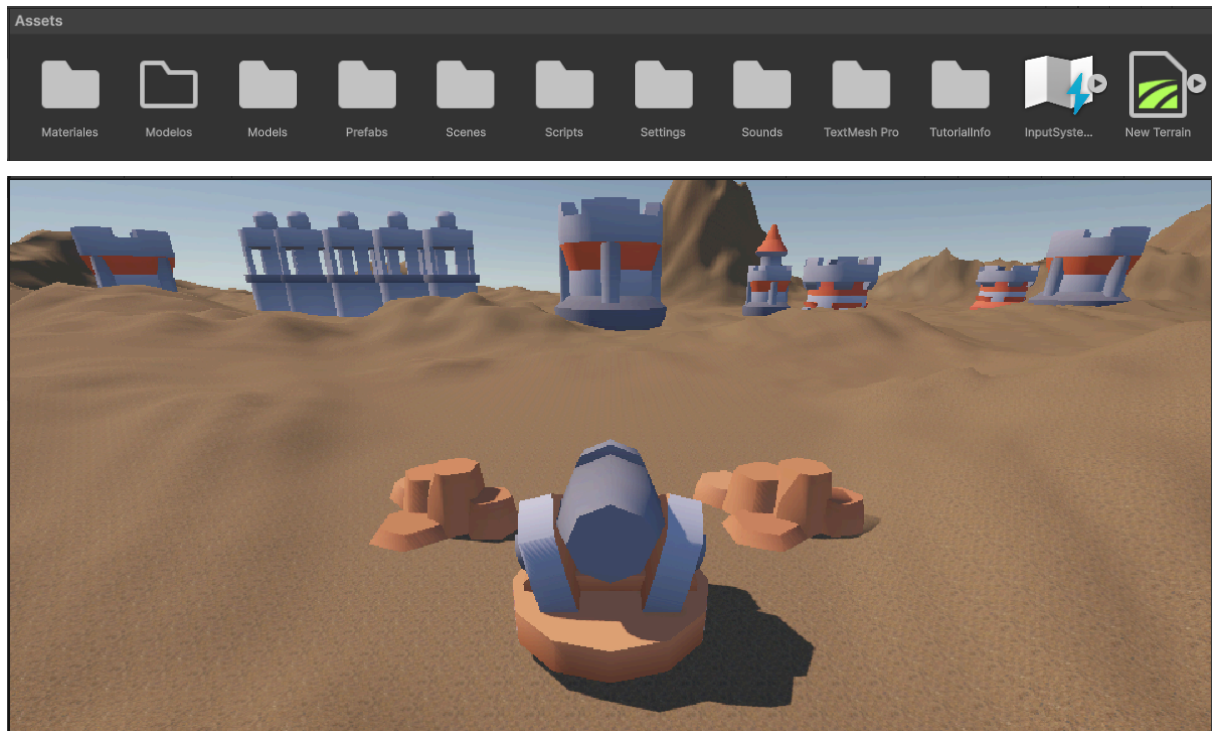
4. Gestión de Colisiones y UI

- 4.1. Detección: Se utilizó `OnCollisionEnter` para detectar impactos. Al colisionar con un "Target", se invoca una función que suma 10 puntos al estado global del juego.
- 4.2. Interfaz de Usuario (UI): Se implementó un Canvas con un panel de victoria que se activa mediante `SetActive(true)` y detiene la simulación física (`Time.timeScale = 0`) para ofrecer una conclusión clara al usuario.



5. Resultados y Conclusiones

La aplicación logra una simulación fluida donde los valores físicos afectan directamente el desplazamiento del objeto. Se mejoró la experiencia del usuario integrando efectos sonoros (música de fondo y sonido de disparo).



6. Referencias:

<https://unity.com/es>

<https://pixabay.com/es/sound-effects/search/guerra/>

<https://pixabay.com/es/sound-effects/search/disparo/>